

Préparation de la soutenance

Mehdi Qostali et Hakim Ziane

Table des matières

1	Modalités	5
2	Objectif du projet	5
3	Différence entre apprentissage supervisé et non-supervisé	6
3.1	Apprentissage supervisé	6
3.2	Comment chaque modèle reconnaît une image	6
3.3	Apprentissage non-supervisé	7
3.4	Comparaison simple	8
4	Précision importante sur le fonctionnement du supervisé	8
4.1	Exemple général	8
4.2	Différence selon les modèles	9
4.2.1	kNN	9
4.2.2	Arbre de décision	9
4.2.3	Forêt aléatoire	9
4.2.4	Perceptron et régression logistique	10
4.3	Formulation correcte à retenir	10
5	Trame orale pour les 4 à 5 minutes	10
5.1	1. Présentation des données, environ 30 secondes	10
5.2	2. Préparation des données, environ 45 secondes	11
5.3	3. Apprentissage supervisé, environ 1 minute 30	11
5.4	4. Analyse des erreurs, environ 45 secondes	11
5.5	5. Apprentissage non-supervisé, environ 1 minute	12
5.6	6. Conclusion, environ 30 secondes	12
6	Répartition possible en binôme	13
6.1	Personne 1	13
6.2	Personne 2	13
7	Questions probables et réponses	13
7.1	Pourquoi normaliser les pixels ?	13

7.2	Pourquoi utiliser un échantillon stratifié ?	13
7.3	C'est quoi la baseline ?	13
7.4	Pourquoi la forêt aléatoire fonctionne mieux ?	13
7.5	Comment lire une matrice de confusion ?	14
7.6	Pourquoi Shirt est difficile ?	14
7.7	Pourquoi utiliser l'ACP avant le clustering ?	14
7.8	Pourquoi $k = 10$ pour k-means ?	14
7.9	Que signifient ARI et NMI ?	14
7.10	Où est utilisé scikit-learn ?	14
8	Questions approfondies possibles du professeur	14
8.1	Pourquoi avoir séparé le code dans le dossier <code>iads</code> au lieu de tout mettre dans le notebook ?	15
8.2	Que fait exactement <code>split_features_labels</code> ?	15
8.3	Pourquoi <code>X</code> a 784 colonnes ?	15
8.4	Pourquoi convertir les pixels en <code>float32</code> ?	15
8.5	Pourquoi les labels sont-ils en <code>int64</code> ?	16
8.6	Pourquoi utiliser <code>random_state=42</code> ?	16
8.7	Quelle est la différence entre validation croisée et test final ?	16
8.8	Pourquoi utiliser <code>StratifiedKFold</code> ?	16
8.9	Que signifie <code>accuracy_moyenne</code> dans la validation croisée ?	16
8.10	Pourquoi regarder aussi l'écart-type ?	17
8.11	Pourquoi la baseline est importante ?	17
8.12	Pourquoi le perceptron et la régression logistique utilisent <code>StandardScaler</code> ?	17
8.13	Quelle est la différence entre perceptron et régression logistique ?	17
8.14	Pourquoi kNN peut être lent ?	17
8.15	Pourquoi limiter la profondeur de l'arbre de décision ?	18
8.16	Pourquoi la forêt aléatoire est souvent meilleure qu'un arbre seul ?	18
8.17	Que signifie <code>n_estimators</code> dans une forêt aléatoire ?	18
8.18	Que signifie <code>n_jobs=-1</code> ?	18
8.19	Pourquoi l'accuracy ne suffit pas toujours ?	18
8.20	Quelle est la différence entre précision et rappel ?	19
8.21	Pourquoi le rappel de <code>Shirt</code> est faible ?	19
8.22	Comment interpréter une case hors diagonale dans la matrice de confusion ?	19
8.23	Pourquoi <code>binary_subset</code> transforme les labels en 0 et 1 ?	19
8.24	Pourquoi avoir choisi T-shirt/top contre Shirt ?	19
8.25	Pourquoi utiliser l'ACP avant k-means ?	20
8.26	Que signifie la variance expliquée par l'ACP ?	20
8.27	Comment fonctionne k-means ?	20
8.28	Que représente l'inertie dans k-means ?	20
8.29	Que mesure le score silhouette ?	20

8.30	Pourquoi ARI et NMI ne sont pas des scores supervisés d'entraînement ?	21
8.31	Pourquoi le clustering ne retrouve pas parfaitement les 10 classes ?	21
8.32	Quelle est la différence entre k-means et clustering hiérarchique ?	21
8.33	Pourquoi le clustering hiérarchique est fait sur moins d'exemples ?	21
8.34	Que fait <code>cluster_label_table</code> ?	21
8.35	Que signifie la pureté d'un cluster ?	21
8.36	Pourquoi vos modèles ne sont pas forcément optimaux pour des images ?	22
8.37	Quelle est la principale limite de votre travail ?	22
8.38	Si vous aviez plus de temps, que feriez-vous ?	22
8.39	Comment savez-vous que le code fonctionne correctement ?	22
8.40	Que se passerait-il si les labels étaient mal séparés des pixels ?	23
8.41	Pourquoi afficher des exemples mal classés ?	23
9	Phrase principale à apprendre	23
10	Explication du dossier iads	23
11	Rôle général	23
12	utils.py	24
12.1	<code>CLASS_NAMES</code>	24
12.2	<code>split_features_labels(dataframe, label_col="label", normalize=True)</code>	24
12.3	<code>stratified_sample(X, y, n_samples, random_state=42)</code>	25
12.4	<code>binary_subset(X, y, class_a, class_b)</code>	25
12.5	<code>class_distribution(y)</code>	26
12.6	<code>plot_images_grid(X, y=None, indices=None, n_cols=8, title=None)</code>	26
12.7	<code>plot_confusion_matrix(matrix, title="Matrice de confusion")</code>	27
13	Classifiers.py	27
13.1	<code>baseline_classifier()</code>	28
13.2	<code>perceptron_classifier(random_state=42)</code>	28
13.3	<code>sgd_logistic_classifier(random_state=42)</code>	28
13.4	<code>knn_classifier(n_neighbors=5)</code>	29
13.5	<code>decision_tree_classifier(random_state=42, max_depth=18)</code>	29
13.6	<code>random_forest_classifier(random_state=42, n_estimators=120, max_depth=None)</code> 29	
14	evaluation.py	30
14.1	<code>cross_validate_classifiers(classifiers, X, y, cv=3, random_state=42)</code> . . .	30
14.2	<code>fit_and_score(model, X_train, y_train, X_test, y_test)</code>	30
14.3	<code>confusion_dataframe(y_true, y_pred, labels, names=None)</code>	31
14.4	<code>classification_report_dataframe(y_true, y_pred, target_names=None)</code>	31

15 Clustering.py	32
15.1 evaluate_kmeans_range(X, y_true, k_values, random_state=42)	32
15.2 cluster_label_table(cluster_labels, true_labels)	33
15.3 agglomerative_labels(X, n_clusters=10, linkage="ward")	33
16 __init__.py	33
17 Résumé ultra-court du code	34
18 Réponse si on demande ce que vous avez codé	34
19 Réponse si on demande la limite principale du projet	34

1 Modalités

- Durée : 10 minutes pour un binôme, 7 minutes pour un monôme.
- Support : ordinateur avec le notebook et le poster.
- Déroulement :
- présentation rapide des expérimentations et résultats : maximum 4 à 5 minutes ;
- questions individuelles sur les expériences et le code Python.
- La note est individuelle, même dans un binôme.

2 Objectif du projet

Le projet porte sur Fashion-MNIST, un jeu de données composé d'images de vêtements.

Le but est double :

- faire de l'apprentissage supervisé pour prédire la classe d'une image ;
- faire de l'apprentissage non-supervisé pour voir si les images se regroupent naturellement en classes cohérentes.

Fashion-MNIST contient :

- 60 000 images d'entraînement ;
- 10 000 images de test ;
- 10 classes ;
- des images en niveaux de gris de taille 28 x 28 pixels.

Chaque image est donc représentée par 784 valeurs numériques.

Répartition des images par classe :

Jeu de données	Total	Nombre de classes	Images par classe
Entraînement	60 000	10	6 000
Test	10 000	10	1 000

Donc, dans le jeu complet, chaque classe possède :

- 6 000 images d'entraînement ;
- 1 000 images de test.

Exemple :

```
label 0 = T-shirt/top    -> 6 000 images train, 1 000 images test
label 6 = Shirt          -> 6 000 images train, 1 000 images test
label 9 = Ankle boot     -> 6 000 images train, 1 000 images test
```

Dans le notebook, certains calculs utilisent des échantillons stratifiés pour réduire le temps de calcul.

Exemples :

- échantillon multi-classe de 5 000 images : environ 500 images par classe ;
- échantillon multi-classe de 20 000 images : environ 2 000 images par classe ;
- classification binaire T-shirt/top vs Shirt : 12 000 images d'entraînement dans le jeu complet, soit 6 000 par classe ;
- test binaire T-shirt/top vs Shirt : 2 000 images, soit 1 000 par classe.

Phrase à retenir :

Dans Fashion-MNIST, chaque classe possède 6 000 images d'entraînement et 1 000 images de test dans le jeu complet.

3 Différence entre apprentissage supervisé et non-supervisé

3.1 Apprentissage supervisé

Dans l'apprentissage supervisé, chaque exemple d'entraînement possède déjà une réponse connue.

Dans notre projet :

- l'entrée est une image de vêtement ;
- la sortie attendue est son label, par exemple **Shirt**, **Trouser** ou **Bag**.

Le modèle apprend donc à associer les pixels d'une image à une classe connue.

Exemple :

- entrée : une image Fashion-MNIST ;
- label connu : **Shirt** ;
- calcul : le modèle apprend les caractéristiques qui permettent de reconnaître cette classe ;
- sortie attendue après entraînement : prédire **Shirt** pour une image similaire.

Les modèles supervisés utilisés dans le projet sont par exemple :

- perceptron ;
- régression logistique avec SGD ;
- kNN ;
- arbre de décision ;
- forêt aléatoire.

3.2 Comment chaque modèle reconnaît une image

Tous les modèles n'utilisent pas la même idée de ressemblance.

Modèle	Principe utilisé
Perceptron	Apprend des poids sur les pixels et choisit la classe avec le meilleur score linéaire.
Régression logistique avec SGD	Apprend aussi des poids, puis estime un score ou une probabilité pour chaque classe.
kNN	Compare directement la nouvelle image aux images d'entraînement les plus proches.
Arbre de décision	Pose une suite de questions sur les valeurs des pixels, puis arrive dans une feuille associée à une classe.
Forêt aléatoire	Combine plusieurs arbres : chaque arbre vote, puis la classe majoritaire est choisie.

Phrase à retenir :

Le kNN mesure directement la ressemblance entre images. Les autres modèles apprennent plutôt des règles numériques à partir des pixels pour choisir la classe la plus probable.

À dire à l'oral :

En supervisé, le modèle apprend avec les bonnes réponses. On lui donne les images et leurs labels, puis on vérifie s'il arrive à prédire correctement les labels sur de nouvelles images.

3.3 Apprentissage non-supervisé

Dans l'apprentissage non-supervisé, le modèle ne reçoit pas les labels pendant l'apprentissage.

Dans notre projet :

- l'entrée est toujours une image de vêtement ;
- mais le modèle ne connaît pas sa vraie classe ;
- il cherche seulement à regrouper les images qui se ressemblent.

Exemple :

- entrée : plusieurs images Fashion-MNIST sans labels ;
- calcul : le modèle mesure les ressemblances entre images ;
- sortie : des groupes, appelés clusters.

Les méthodes non-supervisées utilisées dans le projet sont :

- ACP pour réduire la dimension ;
- k-means pour créer des clusters ;
- clustering hiérarchique pour construire des groupes progressivement.

À dire à l'oral :

En non-supervisé, le modèle n'a pas les bonnes réponses. Il cherche tout seul des groupes d'images similaires. Ensuite seulement, on compare ces groupes aux vraies classes avec des scores comme ARI et NMI.

3.4 Comparaison simple

Point comparé	Supervisé	Non-supervisé
Labels pendant l'apprentissage	Oui	Non
Objectif	Prédire une classe	Trouver des groupes
Exemple dans le projet	Forêt aléatoire	k-means
Sortie	Label prédit	Numéro de cluster
Évaluation	Accuracy, matrice de confusion	ARI, NMI, silhouette

Phrase courte à retenir :

Le supervisé apprend à prédire des labels connus, tandis que le non-supervisé cherche des groupes sans connaître les labels.

4 Précision importante sur le fonctionnement du supervisé

Il ne faut pas confondre l'apprentissage supervisé avec une simple comparaison à une image moyenne ou à une figure représentative.

Dans notre projet, au départ, on sépare bien :

- **X** : les pixels des images ;
- **y** : les labels numériques associés aux images.

Exemple :

```
X[0] = les 784 pixels d'une image  
y[0] = le label réel de cette image, par exemple 6
```

Le label 6 correspond ensuite à la classe lisible **Shirt**.

Mais le modèle n'apprend pas forcément en créant une image représentative de chaque classe.

Le fonctionnement dépend du modèle utilisé.

4.1 Exemple général

Pendant l'entraînement, le modèle reçoit beaucoup d'exemples :

```
pixels image 1 -> label 6  
pixels image 2 -> label 0  
pixels image 3 -> label 9  
...
```

Il cherche ensuite des régularités numériques dans les pixels.

Après entraînement, pour une nouvelle image, il reçoit seulement les pixels :

```
nouvelle image = 784 pixels
```

et il prédit un label :

```
label prédit = 6
```

Puis on traduit ce label avec **CLASS_NAMES** :

6 -> Shirt

Phrase importante :

En supervisé, le modèle apprend à associer des motifs de pixels à des labels connus. Il ne se contente pas forcément de comparer l'image à une image moyenne de chaque classe.

4.2 Différence selon les modèles

Tous les modèles ne raisonnent pas de la même façon.

4.2.1 kNN

Le kNN est le modèle qui ressemble le plus à une comparaison par ressemblance.

Pour une nouvelle image :

- il cherche les images d'entraînement les plus proches ;
- il regarde leurs labels ;
- il prédit la classe majoritaire parmi les voisins.

Phrase à dire :

Avec kNN, la prédiction repose directement sur la ressemblance avec des exemples déjà connus.

4.2.2 Arbre de décision

Un arbre de décision ne compare pas à une image moyenne.

Il pose une suite de questions sur les pixels.

Exemple simplifié :

```
pixel 120 > 0,4 ?  
pixel 315 < 0,2 ?  
pixel 76 > 0,7 ?
```

À la fin du chemin dans l'arbre, il donne une classe.

Phrase à dire :

Un arbre prédit une classe en suivant une suite de décisions sur les valeurs des pixels.

4.2.3 Forêt aléatoire

Une forêt aléatoire combine plusieurs arbres de décision.

Pour une image :

- chaque arbre donne une prédiction ;
- les arbres votent ;
- la classe majoritaire est choisie.

Exemple :

```
arbre 1 -> Shirt
arbre 2 -> Shirt
arbre 3 -> T-shirt/top
arbre 4 -> Shirt
```

Résultat :

classe prédite = Shirt

Phrase à dire :

Dans notre meilleur modèle, la forêt aléatoire prédit grâce au vote de plusieurs arbres, pas grâce à une seule image représentative.

4.2.4 Perceptron et régression logistique

Ces modèles sont linéaires.

Ils apprennent des poids numériques associés aux pixels.

Chaque pixel influence plus ou moins la décision finale.

Phrase à dire :

Les modèles linéaires apprennent des poids sur les pixels pour calculer un score par classe.

4.3 Formulation correcte à retenir

Formulation à éviter :

On crée une figure représentative de chaque classe et on compare la nouvelle image avec cette figure.

Formulation correcte :

On sépare les pixels et les labels. Ensuite, chaque modèle apprend à sa façon une règle qui associe les motifs de pixels aux labels. Pour une nouvelle image, le modèle utilise cette règle pour prédire le label le plus probable.

5 Trame orale pour les 4 à 5 minutes

5.1 1. Présentation des données, environ 30 secondes

Phrase possible :

Nous avons travaillé sur Fashion-MNIST. Le jeu de données contient 60 000 images d'entraînement et 10 000 images de test. Chaque image représente un vêtement en niveaux de gris, avec une taille de 28 x 28 pixels. Il y a 10 classes, par exemple T-shirt/top, Trouser, Pullover, Shirt, Bag ou Ankle boot.

À retenir :

- une image = 784 pixels ;
- un label = la classe réelle du vêtement ;
- les classes sont globalement équilibrées.

5.2 2. Préparation des données, environ 45 secondes

Phrase possible :

Nous avons séparé les pixels et les labels avec la fonction `split_features_labels`. Les pixels ont été normalisés en divisant les valeurs par 255, pour passer de l'intervalle $[0, 255]$ à $[0, 1]$. Nous avons aussi utilisé des échantillons stratifiés pour garder les proportions des classes tout en réduisant le temps de calcul.

À retenir :

- `X` contient les pixels ;
- `y` contient les labels ;
- la normalisation rend les valeurs plus faciles à traiter ;
- l'échantillonnage stratifié évite de déséquilibrer les classes.

5.3 3. Apprentissage supervisé, environ 1 minute 30

Phrase possible :

Nous avons comparé plusieurs modèles supervisés : une baseline, un perceptron, une régression logistique avec SGD, un kNN, un arbre de décision et une forêt aléatoire. La baseline sert seulement de référence minimale. Les modèles ont été comparés avec une validation croisée stratifiée.

Phrase pour le binaire :

En classification binaire, nous avons étudié T-shirt/top contre Shirt. Ce choix est intéressant parce que les deux classes sont visuellement proches. La forêt aléatoire obtient une accuracy de 0,8535 sur le jeu de test.

Phrase pour le multi-classe :

En classification multi-classe, sur les 10 classes, la forêt aléatoire est aussi le meilleur modèle parmi ceux testés. Elle obtient une accuracy de 0,8700 sur le jeu de test.

À retenir :

- la forêt aléatoire est le meilleur modèle testé ;
- le binaire T-shirt/top contre Shirt est difficile ;
- le multi-classe donne un bon score global, mais certaines classes restent confondues.

5.4 4. Analyse des erreurs, environ 45 secondes

Phrase possible :

Nous avons utilisé une matrice de confusion pour comprendre les erreurs. Dans cette matrice, les lignes correspondent aux vraies classes et les colonnes aux classes prédites. La diagonale correspond aux bonnes prédictions, tandis que les cases hors diagonale correspondent aux erreurs.

Phrase sur les résultats :

Les erreurs se concentrent surtout sur les vêtements du haut. La classe Shirt est souvent confondue avec T-shirt/top, Pullover ou Coat. Cela s'explique par la faible résolution des images, car ces vêtements ont des silhouettes proches en 28 x 28 pixels.

À retenir :

- bleu clair = peu d'exemples ;
- bleu foncé = beaucoup d'exemples ;
- diagonale = bonnes prédictions ;
- hors diagonale = erreurs ;
- Shirt est la classe la plus fragile.

5.5 5. Apprentissage non-supervisé, environ 1 minute

Phrase possible :

Pour la partie non-supervisée, nous avons utilisé une ACP avant le clustering. L'ACP permet de réduire la dimension des données tout en gardant l'information principale. Avec 50 composantes principales, on conserve environ 86,6 % de la variance.

Phrase sur k-means :

Nous avons testé k-means avec plusieurs valeurs de k. Comme Fashion-MNIST contient 10 classes, nous avons surtout étudié $k = 10$. Pour $k = 10$, nous obtenons environ 0,385 en ARI et 0,536 en NMI.

Phrase sur le hiérarchique :

Nous avons aussi testé un clustering hiérarchique sur un échantillon plus petit, car cette méthode est plus coûteuse. Il obtient environ 0,430 en ARI et 0,611 en NMI.

À retenir :

- l'ACP réduit la dimension ;
- k-means ne connaît pas les labels pendant l'apprentissage ;
- ARI et NMI servent à comparer les clusters aux vraies classes ;
- le non-supervisé retrouve des groupes visuels, mais pas parfaitement les 10 classes.

5.6 6. Conclusion, environ 30 secondes

Phrase possible :

Pour conclure, l'apprentissage supervisé donne les meilleurs résultats pour prédire les labels, avec la forêt aléatoire comme meilleur modèle testé. Les erreurs restantes concernent surtout les vêtements du haut, car ils sont visuellement proches. Le non-supervisé permet de retrouver certaines familles visuelles, mais il ne reconstitue pas parfaitement les 10 classes.

Phrase de fin :

Pour améliorer le projet, on pourrait entraîner les modèles sur toute la base, optimiser davantage les hyper-paramètres ou tester un modèle convolutionnel, mieux adapté aux images.

6 Répartition possible en binôme

6.1 Personne 1

- présenter Fashion-MNIST ;
- expliquer la préparation des données ;
- expliquer les modèles supervisés ;
- présenter les résultats binaires et multi-classes.

6.2 Personne 2

- expliquer la matrice de confusion ;
- analyser les erreurs ;
- expliquer l'ACP ;
- présenter k-means et le clustering hiérarchique ;
- conclure.

Important : chacun doit quand même savoir répondre sur toute la chaîne du projet, car les questions sont individuelles.

7 Questions probables et réponses

7.1 Pourquoi normaliser les pixels ?

Les pixels sont au départ entre 0 et 255. On les divise par 255 pour obtenir des valeurs entre 0 et 1. Cela rend les données plus faciles à traiter pour certains modèles, surtout les modèles linéaires comme le perceptron ou la régression logistique.

7.2 Pourquoi utiliser un échantillon stratifié ?

Un échantillon stratifié garde les proportions des classes.

Par exemple, si chaque classe représente environ 10 % du jeu complet, elle représente aussi environ 10 % de l'échantillon.

Cela permet de réduire le temps de calcul sans créer un déséquilibre artificiel.

7.3 C'est quoi la baseline ?

La baseline est un modèle très simple qui prédit toujours la classe la plus fréquente.

Elle sert de point de comparaison minimal. Un vrai modèle doit obtenir un meilleur score que cette baseline.

7.4 Pourquoi la forêt aléatoire fonctionne mieux ?

Une forêt aléatoire combine plusieurs arbres de décision.

Chaque arbre vote pour une classe, puis la classe majoritaire est choisie.

Cela permet de réduire les erreurs d'un arbre seul et de mieux capturer des relations non linéaires.

7.5 Comment lire une matrice de confusion ?

Les lignes sont les vraies classes.

Les colonnes sont les classes prédites.

La diagonale contient les bonnes prédictions.

Les cases hors diagonale sont les erreurs.

7.6 Pourquoi Shirt est difficile ?

La classe Shirt ressemble à plusieurs vêtements du haut : T-shirt/top, Pullover et Coat.

Comme les images sont en basse résolution, 28 x 28 pixels, les différences fines sont parfois perdues.

7.7 Pourquoi utiliser l'ACP avant le clustering ?

Les images ont 784 dimensions.

L'ACP réduit cette dimension tout en conservant une grande partie de l'information.

Cela accélère le clustering et peut réduire le bruit.

7.8 Pourquoi $k = 10$ pour k-means ?

Fashion-MNIST contient 10 classes.

Choisir $k = 10$ permet de comparer les 10 clusters obtenus aux 10 vraies classes.

7.9 Que signifient ARI et NMI ?

ARI et NMI sont des scores qui comparent les clusters obtenus avec les vraies classes.

Ils ne sont pas utilisés pour entraîner k-means. Ils servent seulement à évaluer le résultat après coup.

Plus le score est élevé, meilleure est la correspondance entre clusters et classes.

7.10 Où est utilisé scikit-learn ?

Scikit-learn est utilisé pour :

- les modèles supervisés ;
- la validation croisée ;
- la PCA ;
- k-means ;
- le clustering hiérarchique ;
- les métriques comme accuracy, ARI, NMI et matrice de confusion.

8 Questions approfondies possibles du professeur

Cette partie sert à vérifier que vous comprenez vraiment le code et les concepts.

8.1 Pourquoi avoir séparé le code dans le dossier `iads` au lieu de tout mettre dans le notebook ?

Cela rend le notebook plus lisible.

Les fonctions réutilisables sont rangées dans des fichiers séparés :

- `utils.py` pour les données et les affichages ;
- `Classifieurs.py` pour créer les modèles ;
- `evaluation.py` pour les évaluations ;
- `Clustering.py` pour le non-supervisé.

Réponse courte :

Le dossier `iads` permet de structurer le projet et d'éviter de répéter du code dans le notebook.

8.2 Que fait exactement `split_features_labels` ?

Elle transforme le DataFrame lu depuis le CSV en deux objets :

- `X`, qui contient les pixels ;
- `y`, qui contient les labels.

Elle enlève la colonne `label`, garde les 784 pixels, puis normalise les pixels si demandé.

Réponse courte :

Elle sépare les entrées du modèle et les réponses attendues.

8.3 Pourquoi `X` a 784 colonnes ?

Chaque image Fashion-MNIST mesure 28 x 28 pixels.

Donc :

$$28 \times 28 = 784$$

Chaque image est aplatie en un vecteur de 784 valeurs.

Réponse courte :

Une image 28 x 28 devient un vecteur de 784 pixels.

8.4 Pourquoi convertir les pixels en `float32` ?

Les pixels sont utilisés dans des calculs numériques.

`float32` permet :

- de faire la normalisation ;
- de réduire la mémoire utilisée ;
- d'être compatible avec les modèles scikit-learn.

Réponse courte :

On utilise `float32` pour manipuler efficacement des valeurs numériques normalisées.

8.5 Pourquoi les labels sont-ils en `int64` ?

Les labels sont des classes entières : 0, 1, 2, etc.

`int64` est un type entier standard, bien reconnu par NumPy, pandas et scikit-learn.

Réponse courte :

Les labels sont des numéros de classes, donc on les garde comme entiers.

8.6 Pourquoi utiliser `random_state=42` ?

Certains algorithmes utilisent du hasard :

- découpage des données ;
- initialisation de k-means ;
- construction des forêts aléatoires ;
- validation croisée mélangée.

`random_state=42` rend les résultats reproductibles.

Réponse courte :

Cela permet d'obtenir les mêmes résultats à chaque exécution.

8.7 Quelle est la différence entre validation croisée et test final ?

La validation croisée sert à comparer les modèles pendant les expérimentations.

Le test final sert à évaluer le modèle choisi sur des données qui n'ont pas servi au choix du modèle.

Réponse courte :

La validation croisée sert à choisir, le test final sert à mesurer la performance finale.

8.8 Pourquoi utiliser `StratifiedKFold` ?

`StratifiedKFold` garde les proportions des classes dans chaque fold.

C'est important car Fashion-MNIST contient plusieurs classes.

Si un fold contenait trop peu d'une classe, l'évaluation serait moins fiable.

Réponse courte :

On l'utilise pour que chaque fold reste représentatif des 10 classes.

8.9 Que signifie `accuracy_moyenne` dans la validation croisée ?

C'est la moyenne des accuracies obtenues sur les différents folds.

Exemple avec 3 folds :


```
accuracy_moyenne = (score_fold_1 + score_fold_2 + score_fold_3) / 3
```

Réponse courte :

C'est le score moyen du modèle sur plusieurs découpages des données.

8.10 Pourquoi regarder aussi l'écart-type ?

L'écart-type indique si les scores changent beaucoup d'un fold à l'autre.

Un faible écart-type signifie que le modèle est assez stable.

Un fort écart-type signifie que la performance dépend beaucoup du découpage.

Réponse courte :

L'écart-type mesure la stabilité du modèle pendant la validation croisée.

8.11 Pourquoi la baseline est importante ?

La baseline donne un score minimal de référence.

Si un modèle ne fait pas mieux que la baseline, il n'apprend pas vraiment une règle utile.

Réponse courte :

Elle permet de vérifier que nos modèles font mieux qu'une prédiction naïve.

8.12 Pourquoi le perceptron et la régression logistique utilisent StandardScaler ?

Ces modèles sont sensibles à l'échelle des variables.

`StandardScaler` centre et réduit les données.

Cela aide les modèles linéaires à apprendre plus correctement.

Réponse courte :

On standardise parce que les modèles linéaires sont sensibles à l'échelle des variables.

8.13 Quelle est la différence entre perceptron et régression logistique ?

Le perceptron est un modèle linéaire qui cherche une frontière de décision.

La régression logistique estime plutôt des scores liés à des probabilités de classes.

Dans le projet, la régression logistique est entraînée avec `SGDClassifier(loss="log_loss")`.

Réponse courte :

Les deux sont linéaires, mais la régression logistique optimise une perte probabiliste.

8.14 Pourquoi kNN peut être lent ?

kNN compare une image test avec beaucoup d'images d'entraînement.

Il doit calculer des distances au moment de prédire.

Donc la prédiction peut devenir coûteuse si le jeu de données est grand.

Réponse courte :

kNN est lent en prédiction, car il compare les exemples aux données d'entraînement.

8.15 Pourquoi limiter la profondeur de l'arbre de décision ?

Un arbre trop profond peut apprendre les détails du jeu d'entraînement au lieu de généraliser. C'est du sur-apprentissage.

Limiter `max_depth` réduit ce risque.

Réponse courte :

On limite la profondeur pour éviter le sur-apprentissage.

8.16 Pourquoi la forêt aléatoire est souvent meilleure qu'un arbre seul ?

Un arbre seul peut être instable.

Une forêt aléatoire entraîne plusieurs arbres différents, puis combine leurs votes.

Cela réduit les erreurs individuelles.

Réponse courte :

La forêt aléatoire est plus robuste, car elle combine plusieurs arbres.

8.17 Que signifie `n_estimators` dans une forêt aléatoire ?

`n_estimators` est le nombre d'arbres dans la forêt.

Plus il y a d'arbres, plus le vote est stable, mais plus le calcul est long.

Réponse courte :

`n_estimators` indique combien d'arbres participent au vote final.

8.18 Que signifie `n_jobs=-1` ?

Cela demande à scikit-learn d'utiliser tous les coeurs disponibles du processeur.

Le but est d'accélérer certains calculs.

Réponse courte :

`n_jobs=-1` parallélise le calcul sur tous les coeurs disponibles.

8.19 Pourquoi l'accuracy ne suffit pas toujours ?

L'accuracy donne le taux global de bonnes prédictions.

Mais elle ne dit pas quelles classes sont mal reconnues.

Pour cela, il faut regarder :

- le rapport de classification ;
- la matrice de confusion ;
- le rappel par classe.

Réponse courte :

L'accuracy résume tout en un nombre, mais elle cache les erreurs par classe.

8.20 Quelle est la différence entre précision et rappel ?

La précision répond à la question :

Parmi les exemples prédits comme une classe, combien sont vraiment de cette classe ?

Le rappel répond à la question :

Parmi les vrais exemples d'une classe, combien le modèle a-t-il retrouvés ?

Réponse courte :

La précision mesure la qualité des prédictions positives, le rappel mesure la capacité à retrouver une classe.

8.21 Pourquoi le rappel de Shirt est faible ?

Beaucoup de vrais **Shirt** sont prédits comme **T-shirt/top**, **Pullover** ou **Coat**.

Cela diminue le rappel de la classe **Shirt**.

Réponse courte :

*Le rappel de **Shirt** est faible parce que beaucoup de vrais shirts sont classés dans d'autres vêtements du haut.*

8.22 Comment interpréter une case hors diagonale dans la matrice de confusion ?

Une case hors diagonale correspond à une erreur.

Par exemple, ligne **Shirt**, colonne **T-shirt/top** :

*des images réellement **Shirt** ont été prédites comme **T-shirt/top**.*

Réponse courte :

Hors diagonale, ce sont les confusions entre vraies classes et classes prédites.

8.23 Pourquoi `binary_subset` transforme les labels en 0 et 1 ?

Pour obtenir un vrai problème de classification binaire.

La première classe devient 0, la deuxième devient 1.

Cela simplifie l'évaluation et la matrice de confusion.

Réponse courte :

On réencode les deux classes pour travailler avec des labels binaires simples.

8.24 Pourquoi avoir choisi **T-shirt/top** contre **Shirt** ?

Ces deux classes sont visuellement proches.

Cela crée une tâche binaire intéressante, plus difficile qu'une séparation évidente comme **Trouser** contre **Bag**.

Réponse courte :

Ce choix permet d'étudier une confusion réaliste entre deux classes proches.

8.25 Pourquoi utiliser l'ACP avant k-means ?

Les images ont 784 dimensions.

k-means peut devenir plus lent et plus sensible au bruit dans un espace très grand.

L'ACP réduit la dimension tout en gardant une grande partie de l'information.

Réponse courte :

L'ACP rend le clustering plus rapide et plus lisible en gardant l'information principale.

8.26 Que signifie la variance expliquée par l'ACP ?

La variance expliquée indique la quantité d'information conservée par les composantes principales.

Dans le projet, 50 composantes gardent environ 86,6 % de la variance.

Réponse courte :

Cela signifie que la réduction de dimension conserve une grande partie de l'information des images.

8.27 Comment fonctionne k-means ?

k-means cherche à former k groupes.

Il alterne deux étapes :

1. affecter chaque point au centre le plus proche ;
2. recalculer les centres des groupes.

Il répète jusqu'à stabilisation.

Réponse courte :

k-means regroupe les points autour de centres appelés centroïdes.

8.28 Que représente l'inertie dans k-means ?

L'inertie mesure la somme des distances entre les points et leur centre de cluster.

Plus elle est faible, plus les points sont proches de leur centre.

Mais elle diminue presque toujours quand k augmente.

Réponse courte :

L'inertie mesure la compacité des clusters.

8.29 Que mesure le score silhouette ?

Le score silhouette mesure si les points sont proches de leur cluster et éloignés des autres clusters.

Il est généralement compris entre -1 et 1.

Plus il est élevé, plus les clusters sont bien séparés.

Réponse courte :

La silhouette mesure la qualité de séparation des clusters.

8.30 Pourquoi ARI et NMI ne sont pas des scores supervisés d'entraînement ?

K-means n'utilise pas les labels pour apprendre.

ARI et NMI utilisent les labels seulement après le clustering pour comparer les groupes obtenus avec les vraies classes.

Réponse courte :

Les labels servent seulement à évaluer les clusters après coup, pas à les créer.

8.31 Pourquoi le clustering ne retrouve pas parfaitement les 10 classes ?

Les clusters sont basés sur la ressemblance visuelle.

Or certaines classes différentes peuvent se ressembler fortement.

Par exemple, **Shirt**, **T-shirt/top**, **Pullover** et **Coat** peuvent être proches dans l'espace des pixels.

Réponse courte :

Le clustering regroupe par ressemblance, pas forcément selon les labels humains.

8.32 Quelle est la différence entre k-means et clustering hiérarchique ?

k-means demande directement un nombre de clusters **k**.

Le clustering hiérarchique construit progressivement une hiérarchie de groupes.

Dans notre projet, on coupe ensuite cette hiérarchie pour obtenir 10 clusters.

Réponse courte :

k-means cherche directement k groupes, alors que le hiérarchique construit une structure de regroupements successifs.

8.33 Pourquoi le clustering hiérarchique est fait sur moins d'exemples ?

Le clustering hiérarchique coûte plus cher en calcul et en mémoire.

Sur trop d'images, il devient lent.

Réponse courte :

On utilise moins d'exemples parce que le hiérarchique est plus coûteux que k-means.

8.34 Que fait cluster_label_table ?

Cette fonction regarde la composition réelle de chaque cluster.

Elle indique combien d'exemples de chaque vraie classe se trouvent dans chaque cluster.

Elle calcule aussi la classe majoritaire et la pureté.

Réponse courte :

Elle sert à comprendre ce que contient chaque cluster.

8.35 Que signifie la pureté d'un cluster ?

La pureté est la proportion de la classe majoritaire dans un cluster.

Exemple :

cluster de 100 images
70 images sont des Bag
pureté = $70 / 100 = 0,70$

Réponse courte :

La pureté mesure si un cluster contient surtout une seule vraie classe.

8.36 Pourquoi vos modèles ne sont pas forcément optimaux pour des images ?

Les images sont aplaties en vecteurs de pixels.

Cela perd une partie de la structure spatiale : voisinage, formes locales, contours.

Un réseau convolutionnel serait plus adapté pour exploiter cette structure.

Réponse courte :

Nos modèles utilisent les pixels aplatis ; un modèle convolutionnel exploiterait mieux la structure des images.

8.37 Quelle est la principale limite de votre travail ?

Les modèles sont comparés sur des échantillons pour garder un temps raisonnable.

De plus, les hyper-paramètres ne sont pas optimisés de façon exhaustive.

Réponse courte :

La principale limite est le compromis entre temps de calcul et recherche complète des meilleurs paramètres.

8.38 Si vous aviez plus de temps, que feriez-vous ?

Améliorations possibles :

- entraîner sur toute la base ;
- faire une recherche d'hyper-paramètres plus complète ;
- tester un modèle convolutionnel ;
- analyser plus finement les classes confondues ;
- ajouter une validation sur plusieurs tailles d'échantillons.

Réponse courte :

Je testerais surtout un modèle convolutionnel et une optimisation plus poussée des hyper-paramètres.

8.39 Comment savez-vous que le code fonctionne correctement ?

On vérifie plusieurs points :

- les shapes de `X_train`, `X_test`, `y_train`, `y_test` ;
- la distribution des classes ;

- les scores de validation croisée ;
- les scores de test ;
- les matrices de confusion ;
- l’affichage d’exemples bien et mal classés.

Réponse courte :

On vérifie à la fois les dimensions, les distributions, les scores et les erreurs visibles.

8.40 Que se passerait-il si les labels étaient mal séparés des pixels ?

Le modèle pourrait apprendre sur de mauvaises colonnes.

Par exemple, si le label restait dans X , le modèle aurait une information directe sur la réponse.

Cela fausserait complètement les résultats.

Réponse courte :

Il faut bien retirer la colonne `label` de X , sinon l’évaluation peut être faussée.

8.41 Pourquoi afficher des exemples mal classés ?

Cela permet de comprendre visuellement les erreurs.

On peut vérifier si l’erreur est logique, par exemple un vêtement ambigu ou une silhouette proche.

Réponse courte :

Les exemples mal classés aident à interpréter concrètement les erreurs du modèle.

9 Phrase principale à apprendre

Notre résultat principal est que la forêt aléatoire est le meilleur modèle testé. Elle atteint 0,8700 d’accuracy en multi-classe, mais les erreurs restent concentrées sur les vêtements du haut, surtout `Shirt`, car ces classes sont visuellement proches.

10 Explication du dossier `iads`

11 Rôle général

Le dossier `iads` est une bibliothèque locale utilisée par le notebook.

Il évite de mettre toutes les fonctions directement dans le notebook.

Dans le notebook, on importe les modules avec :

```
from iads import utils as ut
from iads import Classifiers as cl
from iads import evaluation as ev
from iads import Clustering as clust
```

Cela signifie :

- `ut` sert à préparer et afficher les données ;
- `cl` sert à créer les modèles ;
- `ev` sert à évaluer les modèles ;
- `clust` sert au clustering ;
- `__init__.py` permet à Python de reconnaître `iads` comme un paquet.

12 `utils.py`

Ce fichier contient les fonctions utilitaires de préparation et d’affichage.

12.1 `CLASS_NAMES`

`CLASS_NAMES` associe chaque label numérique à un nom de classe.

Exemples :

- 0 correspond à `T-shirt/top` ;
- 6 correspond à `Shirt` ;
- 9 correspond à `Ankle boot`.

Cela permet d’afficher des noms lisibles au lieu de simples numéros.

12.2 `split_features_labels(dataframe, label_col="label", normalize=True)`

Cette fonction sépare les pixels et les labels.

Entrée :

- un `DataFrame` contenant une colonne `label` ;
- 784 colonnes de pixels.

Calcul :

- récupère la colonne `label` dans `y` ;
- supprime la colonne `label` pour garder seulement les pixels dans `X` ;
- convertit les pixels en nombres ;
- si `normalize=True`, divise les pixels par 255.

Sortie :

- `X`, tableau de taille `(nombre_images, 784)` ;
- `y`, tableau de taille `(nombre_images,)`.

Exemple :

- entrée : `DataFrame` de taille `(60000, 785)` ;
- sortie : `X.shape = (60000, 784)` et `y.shape = (60000,)`.

À dire à l'oral :

Cette fonction transforme le fichier CSV en deux objets : les données d'entrée X et les labels y .

12.3 stratified_sample(X , y , $n_samples$, $random_state=42$)

Cette fonction prend un sous-échantillon équilibré.

Entrée :

- X , les images ;
- y , les labels ;
- $n_samples$, le nombre d'exemples à garder.

Calcul :

- utilise `train_test_split` avec `stratify=y` ;
- garde les proportions des classes ;
- utilise `random_state` pour obtenir le même tirage à chaque exécution.

Sortie :

- X_sample ;
- y_sample .

Exemple :

- entrée : 60 000 images ;
- $n_samples=5000$;
- sortie : 5 000 images avec une répartition équilibrée des classes.

À dire à l'oral :

On utilise cette fonction pour réduire le temps de calcul sans modifier la proportion des classes.

12.4 binary_subset(X , y , $class_a$, $class_b$)

Cette fonction crée un problème binaire à partir du problème multi-classe.

Entrée :

- toutes les images ;
- tous les labels ;
- deux classes à conserver.

Calcul :

- garde seulement les exemples dont le label vaut `class_a` ou `class_b` ;

- transforme `class_a` en label binaire 0 ;
- transforme `class_b` en label binaire 1.

Sortie :

- `X_bin`, les images filtrées ;
- `y_bin`, les labels binaires.

Exemple :

- `class_a=0` pour T-shirt/top ;
- `class_b=6` pour Shirt ;
- sortie : seulement ces deux classes, avec labels 0 et 1.

À dire à l'oral :

Cette fonction nous permet d'étudier spécifiquement la confusion entre T-shirt/top et Shirt.

12.5 `class_distribution(y)`

Cette fonction compte le nombre d'exemples par classe.

Entrée :

- un tableau de labels.

Calcul :

- compte chaque label avec `value_counts()` ;
- trie les labels ;
- ajoute le nom lisible de chaque classe.

Sortie :

- un DataFrame avec les colonnes `classe`, `nom` et `effectif`.

Exemple :

- entrée : `[0, 0, 1, 6]` ;
- sortie : classe 0 avec effectif 2, classe 1 avec effectif 1, classe 6 avec effectif 1.

À dire à l'oral :

Cette fonction vérifie que les classes sont bien réparties.

12.6 `plot_images_grid(X, y=None, indices=None, n_cols=8, title=None)`

Cette fonction affiche plusieurs images dans une grille.

Entrée :

- `X`, les images sous forme de vecteurs de 784 pixels ;
- `y`, les labels, optionnel ;
- `indices`, les images à afficher ;
- `n_cols`, le nombre de colonnes.

Calcul :

- transforme chaque vecteur de 784 pixels en image 28 x 28 ;
- affiche chaque image avec `imshow` ;
- ajoute le nom de la classe si `y` est donné.

Sortie :

- une figure Matplotlib.

À dire à l'oral :

Cette fonction sert à vérifier visuellement les données avant de lancer les modèles.

12.7 `plot_confusion_matrix(matrix, title="Matrice de confusion")`

Cette fonction affiche une matrice de confusion sous forme de carte de chaleur.

Entrée :

- une matrice de confusion sous forme de DataFrame ;
- un titre.

Calcul :

- affiche la matrice avec la palette `Blues` ;
- ajoute les labels des axes ;
- ajoute une barre de couleur.

Sortie :

- une figure Matplotlib.

À dire à l'oral :

Bleu clair signifie peu d'exemples, bleu foncé signifie beaucoup d'exemples. La diagonale représente les bonnes prédictions.

13 `Classifiers.py`

Ce fichier contient les fonctions qui créent les modèles supervisés.

Important : ces fonctions n'entraînent pas les modèles. Elles renvoient seulement des objets scikit-learn prêts à être entraînés avec `.fit()`.

13.1 baseline_classifier()

Crée un modèle de référence très simple.

Calcul :

- utilise `DummyClassifier(strategy="most_frequent")` ;
- le modèle prédit toujours la classe la plus fréquente.

Utilité :

- sert de comparaison minimale.

À dire à l'oral :

La baseline permet de vérifier que nos vrais modèles apprennent réellement quelque chose.

13.2 perceptron_classifier(random_state=42)

Crée un perceptron avec standardisation.

Calcul :

- applique `StandardScaler()` ;
- entraîne ensuite un `Perceptron`.

Pourquoi standardiser :

- le perceptron est sensible à l'échelle des variables.

À dire à l'oral :

Le perceptron est un modèle linéaire. Il cherche une séparation entre les classes.

13.3 sgd_logistic_classifier(random_state=42)

Crée une régression logistique entraînée avec descente de gradient stochastique.

Calcul :

- applique `StandardScaler()` ;
- utilise `SGDClassifier(loss="log_loss")`.

Interprétation :

- `loss="log_loss"` correspond à une régression logistique.

À dire à l'oral :

C'est un modèle linéaire entraîné efficacement avec SGD.

13.4 `knn_classifier(n_neighbors=5)`

Crée un modèle k plus proches voisins.

Calcul :

- pour une image test, cherche les **k** images d'entraînement les plus proches ;
- prédit la classe majoritaire parmi ces voisins ;
- avec `weights="distance"`, les voisins proches comptent plus.

À dire à l'oral :

Le kNN ne construit pas vraiment de modèle complexe pendant l'entraînement. Il compare surtout les distances au moment de prédire.

13.5 `decision_tree_classifier(random_state=42, max_depth=18)`

Crée un arbre de décision.

Calcul :

- construit une suite de décisions sur les pixels ;
- limite la profondeur avec `max_depth`.

Pourquoi limiter la profondeur :

- pour réduire le sur-apprentissage.

À dire à l'oral :

Un arbre seul peut apprendre trop précisément les données d'entraînement. C'est pour cela qu'on limite sa profondeur.

13.6 `random_forest_classifier(random_state=42, n_estimators=120, max_depth=None)`

Crée une forêt aléatoire.

Calcul :

- crée plusieurs arbres de décision ;
- chaque arbre vote pour une classe ;
- la classe finale est celle qui obtient le plus de votes.

Paramètres importants :

- `n_estimators` = nombre d'arbres ;
- `max_depth` = profondeur maximale ;
- `n_jobs=-1` = utilise tous les coeurs disponibles.

À dire à l'oral :

La forêt aléatoire fonctionne mieux qu'un arbre seul, car elle combine plusieurs arbres et réduit les erreurs individuelles.

14 evaluation.py

Ce fichier contient les outils d'évaluation des modèles.

14.1 cross_validate_classifiers(classifiers, X, y, cv=3, random_state=42)

Cette fonction compare plusieurs modèles par validation croisée.

Entrée :

- un dictionnaire de modèles ;
- les données **X** ;
- les labels **y** ;
- le nombre de folds **cv**.

Calcul :

- crée un **StratifiedKFold** ;
- entraîne chaque modèle sur plusieurs découpages ;
- calcule l'accuracy sur chaque fold ;
- mesure le temps total.

Sortie :

- un **DataFrame** avec :
- le nom du modèle ;
- l'accuracy moyenne ;
- l'écart-type ;
- le temps total ;
- les scores de chaque fold.

À dire à l'oral :

La validation croisée permet de comparer les modèles de manière plus fiable qu'un seul découpage train/test.

14.2 fit_and_score(model, X_train, y_train, X_test, y_test)

Cette fonction entraîne un modèle et le teste.

Calcul :

- lance **model.fit(X_train, y_train)** ;
- mesure le temps d'entraînement ;

- lance `model.predict(X_test)` ;
- mesure le temps de prédiction ;
- calcule l'accuracy.

Sortie :

- un dictionnaire contenant :
- le modèle entraîné ;
- les prédictions ;
- l'accuracy ;
- le temps d'apprentissage ;
- le temps de prédiction.

À dire à l'oral :

Cette fonction sert à évaluer le modèle final sur le vrai jeu de test.

14.3 `confusion_dataframe(y_true, y_pred, labels, names=None)`

Cette fonction construit une matrice de confusion.

Entrée :

- les vraies classes ;
- les classes prédites ;
- l'ordre des labels ;
- les noms des classes.

Calcul :

- compte combien de fois chaque vraie classe est prédite comme chaque classe ;
- transforme le résultat en DataFrame.

Sortie :

- une matrice avec vraies classes en lignes et classes prédites en colonnes.

À dire à l'oral :

Cette matrice permet de voir précisément quelles classes sont confondues.

14.4 `classification_report_dataframe(y_true, y_pred, target_names=None)`

Cette fonction crée un rapport de classification.

Elle calcule :

- `precision` ;

- `recall` ;
- `f1-score` ;
- `support`.

Définitions :

- precision : parmi les exemples prédits dans une classe, proportion correcte ;
- recall : parmi les vrais exemples d'une classe, proportion retrouvée ;
- f1-score : équilibre entre precision et recall ;
- support : nombre d'exemples réels de la classe.

À dire à l'oral :

Le rapport de classification permet d'analyser les performances classe par classe, pas seulement avec l'accuracy globale.

15 Clustering.py

Ce fichier contient les fonctions pour l'apprentissage non-supervisé.

15.1 `evaluate_kmeans_range(X, y_true, k_values, random_state=42)`

Cette fonction teste k-means pour plusieurs valeurs de `k`.

Entrée :

- les données `X`, souvent après ACP ;
- les vraies classes `y_true`, seulement pour l'évaluation ;
- les valeurs de `k` à tester.

Calcul :

- entraîne un k-means pour chaque valeur de `k` ;
- récupère les clusters ;
- calcule :
 - inertie ;
 - silhouette ;
 - ARI ;
 - NMI.

Sortie :

- un DataFrame avec les scores pour chaque `k`.

À dire à l'oral :

Les labels ne servent pas à entraîner k-means. Ils servent uniquement à mesurer après coup si les clusters correspondent aux vraies classes.

15.2 `cluster_label_table(cluster_labels, true_labels)`

Cette fonction décrit la composition des clusters.

Entrée :

- les labels de clusters ;
- les vraies classes.

Calcul :

- construit un tableau croisé ;
- compte les vraies classes dans chaque cluster ;
- trouve la classe majoritaire de chaque cluster ;
- calcule la pureté.

Sortie :

- un DataFrame qui montre la composition réelle des clusters.

À dire à l'oral :

Cette fonction permet de comprendre ce que contient chaque cluster.

15.3 `agglomerative_labels(X, n_clusters=10, linkage="ward")`

Cette fonction applique un clustering hiérarchique agglomératif.

Calcul :

- au départ, chaque exemple est son propre cluster ;
- les clusters les plus proches sont fusionnés progressivement ;
- on coupe ensuite la hiérarchie pour obtenir `n_clusters`.

Avec `linkage="ward"` :

- la méthode cherche à limiter la variance à l'intérieur des clusters.

À dire à l'oral :

Le clustering hiérarchique est intéressant à interpréter, mais il coûte plus cher, donc nous l'avons utilisé sur un échantillon plus petit.

16 `__init__.py`

Ce fichier transforme le dossier `iads` en paquet Python.

Il permet d'écrire :

```
from iads import utils
from iads import Classifiers
from iads import evaluation
from iads import Clustering
```

Il définit aussi `__all__`, c'est-à-dire la liste des modules exportés quand on importe le paquet.

À dire à l'oral :

`__init__.py` sert surtout à rendre le dossier importable comme une bibliothèque locale.

17 Résumé ultra-court du code

Le code suit cette logique :

1. `utils.py` prépare les données.
2. `Classifiers.py` crée les modèles.
3. `evaluation.py` compare et évalue les modèles.
4. `Clustering.py` analyse les regroupements non-supervisés.
5. Le notebook assemble tout pour produire les tableaux, graphes et résultats.

18 Réponse si on demande ce que vous avez codé

Réponse possible :

Nous avons organisé le projet en modules. Les fonctions de `utils.py` préparent les données et affichent les graphes. Les fonctions de `Classifiers.py` créent les modèles scikit-learn. Les fonctions de `evaluation.py` automatisent la validation croisée, le test final et les matrices de confusion. Enfin, `Clustering.py` regroupe les fonctions liées à k-means et au clustering hiérarchique.

19 Réponse si on demande la limite principale du projet

Réponse possible :

La principale limite est que les modèles testés ne sont pas spécifiquement conçus pour les images. Ils utilisent les pixels aplatis en vecteurs. Un modèle convolutionnel pourrait mieux exploiter la structure spatiale des images.